InfLLM: Unveiling the Intrinsic Capacity of LLMs for Understanding Extremely Long Sequences with Training-Free Memory

Chaojun Xiao [*1] **Pengle Zhang** [*1] **Xu Han** [1] **Guangxuan Xiao** [2] **Yankai Lin** [3] **Zhengyan Zhang** [1] **Zhiyuan Liu** [1]
Song Han [2] **Maosong Sun** [1]

Abstract

Large language models (LLMs) have emerged as a cornerstone in real-world applications with lengthy streaming inputs, such as LLM-driven agents. However, existing LLMs, pre-trained on sequences with restricted maximum length, cannot generalize to longer sequences due to the out-of-domain and distraction issues. To alleviate these issues, existing efforts employ sliding attention windows and discard distant tokens to achieve the processing of extremely long sequences. Unfortunately, these approaches inevitably fail to capture long-distance dependencies within sequences to deeply understand semantics. This paper introduces a training-free memory-based method, InfLLM, to unveil the intrinsic ability of LLMs to process streaming long sequences. Specifically, InfLLM stores distant contexts into additional memory units and employs an efficient mechanism to lookup token-relevant units for attention computation. Thereby, InfLLM allows LLMs to efficiently process long sequences while maintaining the ability to capture long-distance dependencies. Without any training, InfLLM enables LLMs pre-trained on sequences of a few thousand tokens to achieve superior performance than competitive baselines continually training these LLMs on long sequences. Even when the sequence length is scaled to $1,024$K, InfLLM still effectively captures long-distance dependencies.

1. Introduction

Recently, large language models (LLMs) have achieved profound accomplishments in various tasks (Brown et al., 2020; Bommasani et al., 2021; Han et al., 2021; Touvron et al., 2023a). Their ability to follow complex instructions shed light on the realization of artificial general intelligence (OpenAI, 2023; Ouyang et al., 2022). With the blooming of LLM-driven applications, such as embodied robotics (Driess et al., 2023; Liang et al., 2023) and agent construction (Park et al., 2023; Qian et al., 2023), enhancing the capability of LLMs to process streaming long sequences become increasingly crucial. For instance, LLM-driven agents are required to process information continuously received from external environments based on all historical memories, necessitating a robust capability for handling long sequences.

Due to limitations caused by unseen lengthy inputs (Han et al., 2023) and distracting noisy contexts (Liu et al., 2023; Tworkowski et al., 2023), most LLMs, pre-trained on sequences with a few thousand tokens, cannot generalize on longer sequences and achieve unsatisfactory performance (Press et al., 2022; Zhao et al., 2023). Common solutions usually involve continually training LLMs on longer sequences but further result in substantial costs (Xiong et al., 2023; Li et al., 2023). Improving the length generalizability of LLMs without further training thus receives extensive attention, trying to make LLMs trained on short sequences directly applicable to long sequences. To this end, some recent works employ sliding windows to discard all distant contexts, ensuring that the sequence length and input noises processed at each step do not exceed the maximum capacity of LLMs (Xiao et al., 2023; Han et al., 2023). Thus, the sliding window mechanism enables effective reading of long sequences but fails to understand them, since it cannot capture the necessary long-distance dependencies among sequence tokens. In summary, existing endeavors are still hard to effectively improve the length generalizability of LLMs at a low cost.

In this paper, we propose a training-free memory-based approach, named InfLLM, for streamingly processing extremely long sequences. InfLLM aims to stimulate the intrinsic capacity of LLMs for capturing long-distance dependencies among massive contexts with limited computational costs. Considering the sparsity of attention score matrices, processing each token typically requires only a small portion

[*]Equal contribution [1]Tsinghua University [2]Massachusetts Institute of Technology [3]Renmin University of China. Correspondence to: Chaojun Xiao <xiaocj20@mails.tsinghua.edu.cn>, Xu Han <hanxu2022@tsinghua.edu.cn>, Zhiyuan Liu <liuzy@tsinghua.edu.cn>.

of its context (Zhang et al., 2023b). We construct an external memory containing distant context information. Only some relevant information from the memory is selected for each computation step and other irrelevant noises are ignored. Owing to this, LLMs can understand whole long sequences with a finite window size and avoid noisy context inputs. However, the vast amount of noisy past contexts in long sequences poses significant challenges to the effective location of relevant units and the efficiency of memory lookup.

To address these challenges, we design a block-level context memory. (1) For effective localization of relevant units, the coherent semantics of each block can more effectively fulfill the requirements for long-sequence reasoning than fragmented tokens. Moreover, we select the semantically most significant tokens from each unit, namely the tokens that receive the highest attention score, as the unit representation. This approach helps avoid the interference of unimportant tokens in the relevance calculation. (2) For efficient memory lookup, the block-level memory unit eliminates the need for per-token, per-head relevance calculations, reducing the computational complexity. Additionally, block-level units ensure contiguous memory access and reduce memory loading costs. Benefiting from this, we design an efficient offloading mechanism tailored for context memory. Considering the infrequent usage of most units, InfLLM offloads all units on CPU memory and dynamically retains the frequently used units on GPU memory, significantly reducing the memory usage. InfLLM do not involve any additional training, and can be directly applied to any LLMs.

To evaluate the effectiveness of InfLLM, we employ Vicuna-7B-v1.5 (Chiang et al., 2023) and Mistral-7B-inst-v0.2 (Jiang et al., 2023) as base models, which are pre-trained on the sequences containing no more than 4K and 32K tokens, respectively. We use two widely-used benchmarks, Longbench (Bai et al., 2023) and ∞-Bench (Zhang et al., 2023a), for evaluation. Especially, the average sequence length in ∞-Bench exceeds 100K tokens, which is challenging for most existing LLMs. The experimental results demonstrate that InfLLM enables the LLMs trained on the sequences containing a few thousand tokens to achieve comparable or even superior performance compared to the methods that continually train LLMs on longer sequences. Moreover, we examine InfLLM on the sequences containing $1,024$K tokens, and InfLLM can still effectively capture long-distance dependencies, demonstrating the potential of InfLLM in scenarios involving long streaming inputs.

2. Related Work

Enabling LLMs to process long sequences has been extensively studied (Dong et al., 2023; Tay et al., 2023; Huang et al., 2023) and can generally be categorized into two main approaches: context length extrapolation and efficient context computation. The former aims to enable LLMs trained on short sequences to process much longer sequences. The latter focuses on enhancing the computational efficiency of attention layers, allowing LLMs to utilize limited resources to process longer sequences. Although the focus of this paper is context length extrapolation, we also detailedly introduce efficient transformers. We also present the relevant works for memory-based models, the source of InfLLM.

Context Length Extrapolation. Due to the high computational and memory requirements, the training of LLMs is often restricted to short sequences. Directly applying LLMs to long sequences will suffer from out-of-domain and distraction challenges caused by lengthy and noisy inputs (Han et al., 2023; Tworkowski et al., 2023). Consequently, context length extrapolation has garnered attention as a method to extend the sequence length for LLMs without incurring additional training. The earliest approaches involve designing new relative positional encoding mechanisms during pre-training (Press et al., 2022; Sun et al., 2023). Subsequent studies mainly focus on the widely-used rotary position embedding (RoPE) (Su et al., 2021), and propose to achieve length extension by interpolating positions to introduce non-integer positions (Chen et al., 2023b; Peng et al., 2023; Jin et al., 2024; Chen et al., 2023a). These works can alleviate the out-of-domain issue from the unseen length, but can not alleviate the distraction challenge of noisy context. To address this, Xiao et al. (2023) and Han et al. (2023) employ the sliding window attention mechanism and directly discard all distant contexts to streamingly read extremely long sequences. However as these models overlook information from distant tokens, they can not capture the long-distance dependencies for long-text understanding. InfLLM constructs an efficient context memory to provide LLMs with relevant context information, which enables LLMs to effectively read and understand extremely long sequences.

Efficient Context Computation. The quadratic computational complexity of the attention layers is a primary factor limiting the lengthy sequence-processing capabilities of LLMs. Thus, numerous scholars have endeavored to design efficient attention mechanisms, including the utilization of sparse attention (Zaheer et al., 2020; Beltagy et al., 2020; Child et al., 2019; Ainslie et al., 2020; Zhao et al., 2019), approximating attention computations using kernel functions (Kitaev et al., 2020; Wang et al., 2020; Katharopoulos et al., 2020), and replacing the attention layer with linear-complexity state-space models (Gu et al., 2022; Gu & Dao, 2023). These approaches necessitate a modification in the model architecture, requiring retraining the models. Simultaneously, many researchers have approached this from an infrastructural perspective, optimizing the memory footprint of attention computations to reduce the computational resource demands of the model (Dao et al., 2022; Dao, 2023; Hong et al., 2023; Shazeer, 2019; Kwon et al., 2023). These

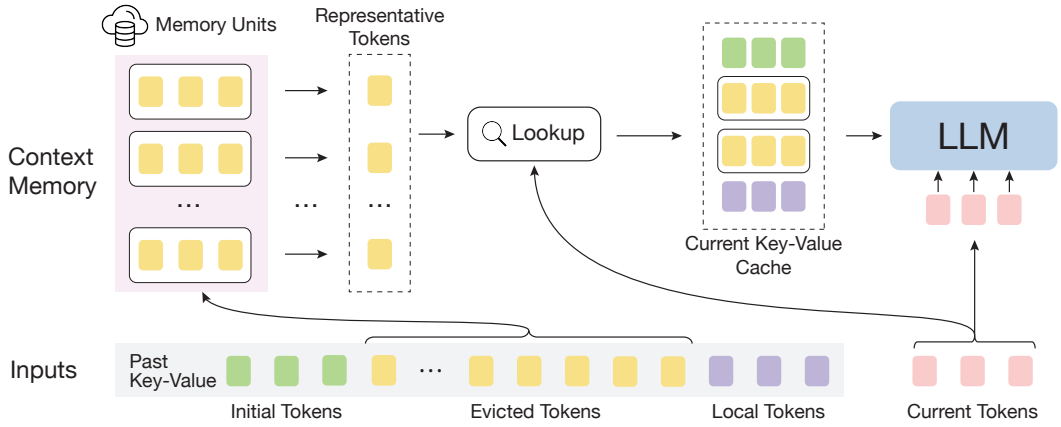


Figure 1: The illustration of InfLLM. Here, the current tokens refer to tokens that need to be encoded in the current computation step. The past key-value hidden states can be divided into the initial tokens, evicted tokens, and local tokens, arranged the furthest to the nearest relative to the current tokens. For each computation step, the past context window consists of the initial tokens, relevant memory units, and local tokens.

works are parallel to ours, and can be directly combined to further accelerate LLM inference.

Memory-based Models. Memory networks have been studied for decades, which are proven effective in providing models with additional knowledge and information storage capabilities (Graves et al., 2014; Weston et al., 2015; Sukhbaatar et al., 2015; Miller et al., 2016). With the success of pre-trained models, memory layers have also been gradually applied in the training processes of recurrent transformer layers, enabling models to process long sequences recursively (Dai et al., 2019; Rae et al., 2020; Khandelwal et al., 2020; Wu et al., 2022; Bertsch et al., 2023). These works split sequences into segments, encoding each segment individually, and use memory to store context information from preceding segments. While these approaches are similar in concept to InfLLM, they involve modifications to the model architecture and should be applied during the pre-training phase. In contrast, we aim to explore the inherent characteristics of LLMs, and propose a training-free memory module for long-text understanding.

3. Methodology

As shown in Figure 1, InfLLM builds a training-free context memory to efficiently provide relevant context information, which endows sliding window attention with the ability to capture long-distance dependencies.

3.1. Overall Framework

The main restrictions for extending the context window of LLMs come from the out-of-domain and distraction issues caused by the lengthy and noisy context. To address the challenges, following previous works (Xiao et al., 2023; Han et al., 2023), we adopt the sliding window attention mechanism, which only considers limited tokens for each step. Additionally, we propose to build an extra context memory module to provide relevant context information to capture long-distance dependencies. Specifically, we denote the long input sequence as $s = \{t_i\}_{i=1}^{l}$. Due to the limited GPU memory, instead of encoding the whole s at once, we encode the input sequence s chunk-by-chunk and generate the output token-by-token. For each computation step, the inputs consist of past key-value vectors $\mathbf{P} = \{(\mathbf{k}_j, \mathbf{v}_j)\}_{j=1}^{l_P}$ and current tokens hidden vectors $\mathbf{X} = \{\mathbf{t}_{i+l_P}\}_{i=1}^{l_X}$. For encoding steps, l_X equals the chunk size, and for decoding steps, l_X equals one.

According to the distances from current tokens, we can divide $\mathbf{P}$ into three groups: the initial tokens, $\mathbf{I} = \mathbf{P}_{[1:l_I]}$, the evicted tokens, $\mathbf{E} = \mathbf{P}_{[l_I+1:l_P-l_L]}$, and local tokens, $\mathbf{L} = \mathbf{P}_{[l_P-l_L+1:l_P]}$, arranged from the furthest to the nearest relative to the current tokens. Here, l_P, l_I, l_L refer to the length of past key-value vectors, initial tokens, and the local window size. All evicted tokens, $\mathbf{E}$, are stored in the context memory, consisting of multiple memory units. For each step, InfLLM concatenates the initial tokens, relevant memories units from context memory, and local tokens to form the current key-value cache, $\mathbf{C} = \text{Concat}(\mathbf{I}, f(\mathbf{X}, \mathbf{E}), \mathbf{L})$. $f(\cdot)$ refers to the lookup operation of context memory. The attention output is calculated as:

$$\mathbf{O} = \text{Attn}\left[\mathbf{QX}, \text{Concat}(\mathbf{C}_k, \mathbf{KX}), \text{Concat}(\mathbf{C}_v, \mathbf{VX})\right].$$

Here, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are parameters in attention layers, $\mathbf{C}_k$ and $\mathbf{C}_v$ refer to the key and value vectors in $\mathbf{C}$. If $f(\cdot)$ always returns empty sets, InfLLM is degenerated into LM-Infinite (Han et al., 2023) and Streaming-LLM (Xiao et al., 2023), which directly discards distant context.

3.2. Context Memory

Previous findings indicate that the attention score matrices of LLMs are sparse, and we can generate the same outputs with only a small portion of key-value vectors preserved (Zhang et al., 2023b). Inspired by this, we design a context memory to efficiently look up relevant contexts from large-scale evicted tokens and ignore irrelevant ones to save computational costs. The most intuitive way is to construct a memory consisting of token-level memory units for every past key-value hidden states, and every attention head separately, which would result in massive memory units and unacceptable computation and non-contiguous memory access costs. Thus, considering the local semantic coherence of long sequences, we split the past key-value hidden states into blocks, each serving as a memory unit, and conduct memory lookup at the block level to reduce the costs while preserving the performance.

In this subsection, we will introduce the details of the block-level memory units. Then we present the method to assign positional embeddings for selected relevant memory units and cache management for the context memory.

Block-Level Memory Units. Block-level memory units can save computation costs compared to token-level ones. It also poses new challenges for unit representations, which are supposed to contain the semantics of the entire unit for effective relevance score computation and be memory-efficient for context length scalability. Traditional methods usually involve training an additional encoder to project a given unit into a low-dimension vector. Inspired by the token redundancy in hidden states (Goyal et al., 2020; Dai et al., 2020), we select several **representative tokens** from the entail blocks as the unit representation. Specifically, for the m-th token, we define the representative score as:

$$r_m = \frac{1}{l_L} \sum_{j=1}^{l_L} \mathbf{q}_{m+j} \cdot \mathbf{k}_m, \tag{1}$$

where $\mathbf{q}_{m+j}$ is the query vector for $(m+j)$-th token and $\mathbf{k}_m$ is the key vector m-th token. Intuitively, r_m represents the significance of the m-th token in its corresponding local window, indicating the extent of its influence on other tokens within the local window. The computation of representative scores requires no additional parameters.

Formally, given the evicted tokens, $\mathbf{E}$, we split it into several memory units, each containing l_{bs} tokens. For each unit, the r_k tokens with the highest representative scores are selected as representative tokens. Generally, r_k is a small positive integer. Let us denote a memory unit as $\mathbf{B} = \{(\mathbf{k}_j^B, \mathbf{v}_j^B)\}_{j=1}^{l_{bs}}$, and the representative tokens of this unit as $R(\mathbf{B}) = \{(\mathbf{k}_{b_j}^B, \mathbf{v}_{b_j}^B)\}_{j=1}^{r_k}$.

For the **memory lookup** phrase, only k_m units with the highest relevance scores are loaded for the current attention computation step. We calculate the relevance score between $\mathbf{B}$ and current tokens $\mathbf{X}$ as:

$$\mathrm{sim}(\mathbf{X}, \mathbf{B}) = \sum_{i=1}^{l_X} \sum_{j=1}^{r_k} \mathbf{q}_{i+l_P} \cdot \mathbf{k}_{b_j}^B. \tag{2}$$

Notably, the representative tokens selection is a training-free method to obtain the unit representations. Here, we can also train an additional encoder to generate more expressive unit representations, which we leave for future work.

Positional Encoding. Existing LLM training usually employs a finite number of positional encodings, which encounter out-of-domain distribution challenges when directly applied to longer sequence processing (Han et al., 2023). Besides, in InfLLM, the current key-value cache is composed of some discontinuous text blocks, and directly assigning continuous positional encodings to them would also lead to mismatch issues and confuse the model. Therefore, inspired by previous works (Raffel et al., 2020; Su, 2023), we assign all tokens beyond the local window size with the same positional encodings. Specifically, the distance between tokens in context memory units and current tokens is set as l_L.

Cache Management. To enable LLMs to process extremely long sequence streams while capturing the semantic relevance contained in the long context, we need to retain all memory units and look up them at each computation step. Considering the infrequent usage of most units, we employ an offloading mechanism, storing most memory units in CPU memory and only preserving the representative tokens and memory units needed in current steps in GPU memory. Additionally, given the semantic coherence of long sequences, where adjacent tokens often require similar memory units, we allocate a cache space in GPU memory, managed using a least recently used strategy. This approach allows for efficient encoding of extremely long sequences using limited GPU memory. From the observation, we find that the miss rate of our GPU cache is quite low, which means the offloading mechanism does not introduce significant time overhead in memory loading while saving GPU memory usage. The details can be found in Appendix.

Furthermore, for extremely long sequences, the representative tokens of each unit can also be offloaded to the CPU memory, constructing an efficient k-nearest-neighbor index, and thereby further reducing computational complexity.

3.3. Connection to Retrieval-Augmented Generation

InfLLM leverages the intrinsic capacity of LLMs to construct a context memory for gathering token-relevant information, a concept similar to retrieval augmented generation (RAG) (Lewis et al., 2020; Nakano et al., 2021). However, compared to using RAG, where historical contexts are treated as a searchable database for long-sequence under-

Table 1: The results of Mistral-based models. The models that can process extremely long streaming inputs are denoted with ♣. The models with continual training are denoted with *. The 95% quantile for text lengths in these two benchmarks is 214K and 31K. The context window size for InfLLM is denoted as "local window size + selected memory size".

		Mistral-7B-inst-v0.2 (32K)							
		Original	Yarn*	PI	NTK	Infinite♣	Stream♣	InfLLM♣	InfLLM♣
	Context Window	32K	128K	128K	128K	6K	6K	4K + 2K	4K + 8K
	Attention Type	Full	Full	Full	Full	Window	Window	Window	Window
∞-Bench (214K tokens)	Math.Find	22.29	17.14	0.00	26.29	14.00	13.71	**26.57**	26.86
	En.MC	44.98	27.95	0.00	37.99	38.86	37.99	**43.23**	41.92
	Code.Debug	35.28	22.59	16.24	28.17	28.93	27.66	**29.44**	36.04
	Retrieve.KV	16.60	0.00	0.00	21.20	3.40	3.40	**95.60**	98.20
	Retrieve.Number	28.81	56.61	0.00	86.27	6.78	6.78	**99.83**	99.32
	Retrieve.PassKey	28.81	92.71	0.00	100.00	6.78	6.78	**100.00**	100.00
	Average	29.46	36.17	2.80	49.99	16.46	16.05	**65.78**	67.06
LongBench (31K tokens)	NarrativeQA	20.66	19.67	23.47	22.67	18.44	17.92	**22.12**	23.03
	Qasper	29.14	11.10	29.36	29.90	30.02	**30.05**	29.33	29.52
	MultiFieldQA	47.39	35.06	46.47	46.41	39.05	39.09	**47.42**	47.62
	HotpotQA	37.60	11.94	37.33	35.18	32.02	32.18	**36.56**	39.53
	2WikiMQA	22.54	12.02	22.19	21.71	22.27	21.83	**22.31**	23.61
	Musique	18.50	7.52	18.80	19.45	15.81	14.71	**17.68**	18.92
	GovReport	31.03	29.46	32.36	32.86	29.74	29.83	**31.03**	31.37
	QMSum	23.84	21.53	23.58	23.16	21.92	21.94	**23.49**	23.77
	MultiNews	26.65	16.04	26.62	26.69	26.65	26.64	**26.70**	26.66
	TREC	71.00	68.50	71.50	71.50	**70.00**	**70.00**	69.00	71.00
	TriviaQA	86.22	88.21	88.86	89.31	85.22	85.57	**86.67**	87.34
	SAMSum	42.11	26.52	42.58	41.55	41.60	41.31	**42.52**	41.80
	PassageRetrieval	89.02	16.25	89.10	92.42	42.80	42.17	**64.00**	87.42
	LCC	57.33	66.39	55.90	55.07	**57.12**	55.38	56.67	56.69
	RepoBench-P	54.27	55.82	52.46	50.33	**53.43**	51.46	52.97	52.09
	Average	43.82	32.40	44.04	43.88	39.07	38.67	**41.90**	44.02

standing (Xu et al., 2023), InfLLM has several advantages: (1) Training-Free: RAG requires additional retrieval data to train a retrieval model, whereas InfLLM is training-free and applicable to any LLMs. Besides, RAG also necessitates fine-tuning LLMs to adapt to the inputs augmented by the retrieved knowledge. (2) Broader Applicability: RAG necessitates the explicit specification of a query for the retrieval model. Therefore, RAG is typically suitable for tasks with a specified query, such as question answering, or requires the construction of an additional query generator. However, for many tasks, it is not feasible to construct an appropriate query to retrieve information from the inputs, such as in summarization. In contrast, InfLLM has no specific requirements for the tasks and can be feasibly used for long sequences. Therefore, RAG is more suited for knowledge-driven tasks to acquire additional external knowledge. Notably, our methods can be directly combined with RAG to advance knowledge-driven tasks. RAG is employed to retrieve external knowledge from the Internet, and the retrieved multiple documents are concatenated as long inputs for InfLLM to generate high-quality answers.

4. Experiments

4.1. Datasets

We adopt representative tasks in two widely-used long document benchmarks, ∞-Bench (Zhang et al., 2023a) and LongBench (Bai et al., 2023) for evaluation. We adopt the English datasets for evaluation as the base models are mainly pre-trained on English corpus. The datasets in ∞-Bench and LongBench cover diverse tasks including question answering, summarization, few-shot learning, context retrieval, mathematic computing, and code completion. The average length for the two benchmarks is 145.1 and 12.8K respectively. The 95% quantile for sequence lengths in these two benchmarks is 214K and 31K, which is far beyond the maximum length of the base models. Detailed statistics and task descriptions of these datasets are listed in the Appendix.

4.2. Baseline Models

In this paper, we aim to enable LLMs trained with limited sequence length to read and understand extremely long sequences without further training. We adopt Mistral-7B-Instruct-v0.2 (Jiang et al., 2023) and Vicuna-7B-v1.5 (Chi-

Table 2: The results of Vicuna-based models. The models that can process extremely long streaming inputs are denoted with ♣. The models with continual training are denoted with *. The 95% quantile for text lengths in these two benchmarks is 214K and 31K. The context window size for InfLLM is denoted as "local window size + selected memory size".

		Vicuna-7B-v1.5 (4K)							
		Original	LChat*	Vic-16K*	PI	NTK	Infinite♣	Stream♣	InfLLM♣
	Context Window	4K	32K	16K	128K	128K	4K	4K	2K + 2K
	Attention Type	Full	Full	Full	Full	Full	Window	Window	Window
∞-Bench (214K tokens)	Math.Find	11.71	9.43	13.43	OOM	OOM	5.71	6.00	**11.14**
	En.MC	30.13	24.45	34.06	OOM	OOM	30.57	**32.31**	31.44
	Code.Debug	38.83	27.66	35.03	OOM	OOM	44.92	**46.19**	34.26
	Retrieve.KV	1.40	1.40	1.00	OOM	OOM	0.00	0.00	0.60
	Retrieve.Number	4.41	23.90	10.34	OOM	OOM	4.24	4.41	**81.69**
	Retrieve.PassKey	5.08	28.64	15.25	OOM	OOM	5.08	4.92	**99.15**
	Average	15.26	19.25	18.19	–	–	15.09	15.64	**43.05**
LongBench (31K tokens)	NarrativeQA	11.19	20.35	17.85	0.78	5.66	14.27	**15.61**	15.53
	Qasper	13.79	29.35	25.85	2.71	21.17	22.88	**23.84**	23.57
	MultiFieldQA	22.08	42.55	37.15	1.01	36.76	32.48	32.80	**37.14**
	HotpotQA	12.71	33.19	24.72	1.35	19.54	22.05	22.17	**22.53**
	2WikiMQA	13.99	24.33	21.41	1.17	14.51	18.13	18.38	**18.82**
	Musique	4.81	14.71	8.44	0.71	4.30	**7.37**	6.30	5.24
	GovReport	27.67	30.83	27.62	1.9	25.26	23.44	23.18	**26.79**
	QMSum	19.72	22.93	22.63	1.29	19.48	20.67	20.09	**20.91**
	MultiNews	26.61	26.63	27.88	1.16	25.88	26.10	26.19	**26.43**
	TREC	69.00	66.50	69.00	4.50	59.00	60.50	61.00	**67.50**
	TriviaQA	81.94	83.99	85.63	0.90	25.85	77.61	78.81	**84.36**
	SAMSum	35.12	12.83	9.15	0.12	5.05	**32.55**	32.46	31.89
	PassageRetrieval	9.00	30.50	4.00	0.62	5.00	7.00	6.00	**9.00**
	LCC	64.53	54.79	50.64	21.54	53.65	**64.22**	63.70	61.41
	RepoBench-P	50.17	58.99	44.94	19.36	44.58	47.18	**48.26**	47.52
	Average	30.82	34.70	31.79	3.94	24.38	31.76	31.92	**33.24**

ang et al., 2023) as our base models. Mistral-7B-Instruct-v0.2 is first pre-trained with a maximum sequence length of 8K and then fine-tuned with a maximum sequence length of 32K. Vicuna-7B-v1.5 is fine-tuned from LLaMA2-7B (Touvron et al., 2023b) with 4K as the maximum length.

To verify the effectiveness of our proposed method, we compare InfLLM with the following competitive baseline models: (1) **Original** is the original LLMs without context length extrapolation. (2) **Position Interpolation (PI)** (Chen et al., 2023b) directly down-scale the position indices for models with RoPE-based position encoding. Thus, within the same maximum position index, we can encode more tokens. (3) **NTK-Aware Scaled RoPE (NTK)** [1] designs a nonlinear interpolation method, which basically changes the rotation base of RoPE. (4) **Continual Training** refers to methods that continually pre-train or fine-tune LLMs with additional long sequences. Here, we choose Yarn-Mistral-128k (Yarn) (Peng et al., 2023) for Mistral-based models. We choose LongChat-32K (LChat) (Li et al., 2023) and Vicuna-16K (Vic-16K) [2] for Vicuna-based models as baselines. These models are fine-tuned versions of scaled RoPE methods. (5) **Sliding Window** indicates the models that apply the sliding window mechanism to discard distant contexts, including LM-Infinite (Infinite) (Han et al., 2023) and StreamingLLM (Stream) (Xiao et al., 2023).

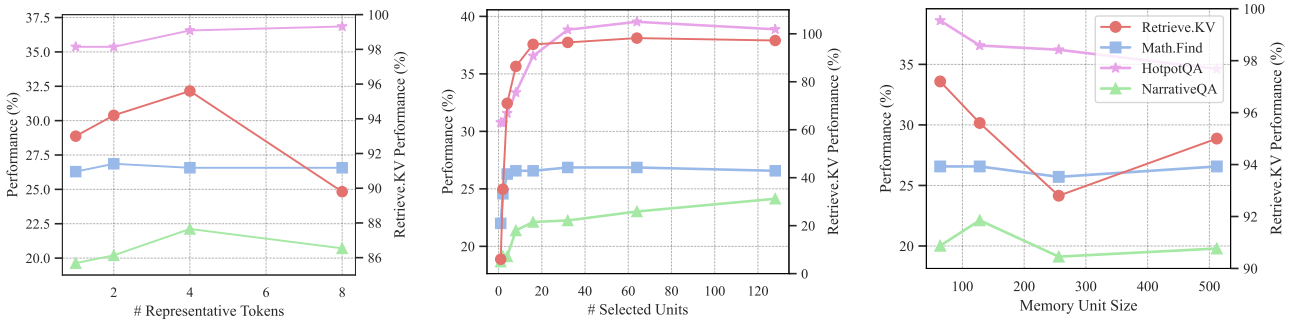
All models except for models with continual training require no additional training. For PI and NTK, we extend the context window to 128K, which enables LLMs to process most instances in both two benchmarks. For models with finite context length, we truncate the inputs by only preserving the system prompts and the tail of inputs to simulate real-world applications with streaming inputs.

4.3. Implementation Details

For our model, we set the encoding chunk size as 512, and the memory unit size for past key-value vectors, l_{bs}, as 128. We load 16 relevant memory units for each step. For Mistral-based InfLLM, we set the local window size as 4K, and for Vicuna-based InfLLM, we set the local window size

[1] https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have/

[2] https://huggingface.co/lmsys/vicuna-7b-v1.5-16k



(a) (b) (c)

Figure 2: Extra studies about InfLLM. Here, (a), (b), and (c) investigate the impact of the context memory under different numbers of representative tokens, different numbers of selected units, and memory unit sizes, respectively.

as 2K. The number of representative tokens, r_k, is set as 4. The number of initial tokens is set as 128 for LM-Infinite, StreamingLLM, and InfLLM to cover the system prompts and task descriptions. We adopt FlashAttention (Dao, 2023) to accelerate experiments for all baseline models. Please refer to the Appendix for more details.

4.4. Main Results

The results for Mistral-based models and Vicuna-based models are reported in Table 1 and Table 2 respectively. From the results, we can observe that: (1) Compared to models with the sliding window mechanism, which can also read extremely long sequences, our method demonstrates a significant performance improvement on two benchmarks. This indicates that the context memory in InfLLM can accurately supplement LLMs with relevant contextual information, enabling efficient and effective understanding and reasoning on long sequences. (2) Under the training-free setting, previous context length extrapolation models, such as PI and NTK, tend to compromise model performance while extending the sequence length to 128K. That is because these models cannot address the distraction issue caused by noisy contexts. In contrast, our model can consistently enhance performance for extremely long sequences. We successfully extend Vicuna from a 4K length to more than 50 times its length, achieving commendable performance on the ∞-Bench. (3) The scaled RoPE methods, including PI, NTK, and continually trained models, can increase the maximum sequence length of LLMs but also raise the computational and memory costs. This leads to Vicuna-based PI and NTK models exceeding GPU memory limits when processing 128K-length sequences, limiting these methods' application to encoding extremely long sequences. In contrast, InfLLM utilizes block-level memory and offloading mechanisms, enabling efficient processing of extremely long sequences within limited resources. (4) When the sequence length is within the model's processing capability, as with Mistral-based models on LongBench, InfLLM can achieve comparable or even superior results with a shorter window size, significantly reducing computational overhead for inference. (5) Compared to models that have undergone continual training on long sequences, InfLLM can achieve comparable or even superior results without any additional training. This suggests that LLMs inherently possess the capability to identify key information in long sequences and to understand and reason effectively.

4.5. Extra Investigation

InfLLM relies on the context memory to look up relevant information. We further explore the impact of core components in the context memory, specifically the representative tokens and memory units. The results are shown in Figure 2.

Different Number of Representative Tokens. InfLLM splits key-value vectors into memory units and selects several representative tokens from the unit to serve as the unit representations. Consequently, the ability of these representative tokens to semantically represent the entire unit directly impacts the model's performance. We conduct experiments with the number of representative tokens as $\{1, 2, 4, 8\}$. The results are shown in Figure 2a. It is observed that as the number of representative tokens increases, there is a trend of improvement in the model performance, which indicates that more representative tokens tend to better represent the semantic content of the memory units. However, it is noted that when the number of representative tokens reaches 8, there is a slight performance decrease on Retrieve.KV and NarrativeQA. This decline can be attributed to the inclusion of semantically irrelevant tokens as unit representations. More efficient and powerful unit representations will further enhance model performance for future work.

Different Number of Selected Units. The selected units are utilized to provide relevant context to LLMs. We conduct experiments with the number of units set as

Table 3: The results for ablation study. Here R.KV, M.Find, NQA, and HQA refer to Retrieve.KV, Math.Find, NarrativeQA, and HotpotQA.

Task	R.KV	M.Find	NQA	HQA
InfLLM	**95.60**	26.57	36.56	**22.12**
Decoding-Only	32.60	26.57	32.74	19.72
w/o Lookup	3.40	14.00	32.02	18.44
Mean Repr	75.20	**26.86**	**37.12**	19.09

$\{1, 2, 4, 8, 32, 64, 128\}$. From Figire 2b, we can observe that as the number of selected units increases from 1 to 16, the model performance significantly improves, which is attributed to that more units imply a greater recall rate of relevant content. Larger unit quantity also leads to an increase in the required memory scheduling time and the computational time for attention. Therefore, further enhancing lookup accuracy remains a crucial direction for improving the efficiency of InfLLM.

Different Memory Unit Size. Each memory unit is supposed to be a coherent semantic unit. Excessively large unit sizes can hinder precise lookup, while a small size will increase the computational overhead of memory lookup. We evaluate InfLLM with the unit size as $\{64, 128, 256, 512\}$ and keep the total context length as 2048. The results are shown in Figure 2c. It can be observed that the optimal unit size varies for different tasks due to the varying characteristics of input sequences. For example, in Retrieve.KV, a key-value pair constitutes a semantic unit, while in Math.Find, a single number represents a semantic unit. Employing heuristic rules to segment context can easily lead to suboptimal performance. Therefore, exploring how to dynamically segment context is an important direction for future research.

4.6. Ablation Study

To further verify the effectiveness of dynamic multi-step memory lookup and unit representations, we conduct ablation studies in this section. The results are shown in Table 3.

Context Memory Lookup. InfLLM adopts dynamic context memory lookup for both input encoding and output decoding steps for comprehensive long-text understanding. We present the results of InfLLM with only lookup in output decoding (Decoding-Only) and without any memory lookup (w/o Lookup). It can be observed that a significant decline in model performance is associated with a reduction in the number of memory lookup iterations. This indicates that distant contextual information is crucial for both the long-input encoding and answer-generation phases. The model requires the integration of long-distance context to generate a coherent context memory for input understanding. LLM is supposed to collect useful information from massive past context information to generate the correct answers.

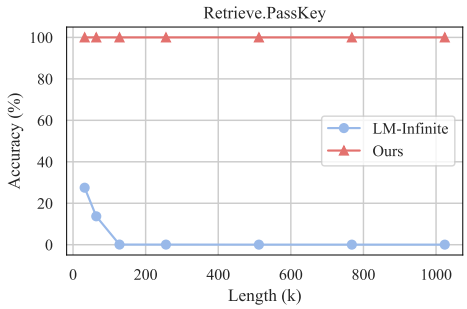


Figure 3: The results for InfLLM and LM-Infinite on sequences with different lengths.

Unit Representation. We design a block-level memory for efficient context information lookup. We select several representative tokens as the unit representations for relevance computation. We present the results of InfLLM with another training-free representation method (Mean Repr), which computes the representation by averaging the key vectors in a memory unit. For a fair comparison, we set the number of representation vectors as 4. From the results, we can observe that InfLLM with average representations can also present competitive performance, and achieve superior results to representative tokens on Math.Find and NarrativeQA. It indicates that the original attention vectors in LLMs are effective for relevance score computation, and exploring more efficient unit representations is an important future direction.

4.7. Scaling to 1,024K Context

To assess the effectiveness of InfLLM on extremely long sequences, in this subsection, we scale the sequence length to 1024K to evaluate the capacity of InfLLM to capture contextual relevance in long sequences. Specifically, we adopt the Retrieve.PassKey task in ∞-Bench for evaluation. This task prompts LLMs to find a 5-digit sequence among lengthy and noisy contexts, which requires LLMs to locate relevant information among long sequences effectively. We automatically generate inputs with $\{32, 64, 128, 256, 512, 768, 1024\}$ thousand tokens and for each length, we generate 50 instances for evaluation. We adopt Mistral as the base model.

The results are shown in Figure 3. From the results, we can observe that InfLLM can accurately locate the key information from length noises and achieve 100% accuracy even when the context length scales to 1024 thousand tokens. However, LM-Infinite can only attend to the tokens within the local window, which leads to a rapid decline in its performance as the sequence length increases. It proves that InfLLM can accurately capture the long-distance dependencies for effective long-sequence reasoning.

5. Conclusion

In this paper, we propose a training-free method to extend the context window of LLMs. Based on the sliding window attention mechanism, we construct an additional context memory module, which can help LLMs select relevant information from massive contexts to capture long-distance dependencies. The experiments on two widely-used long-text benchmarks show that InfLLM can effectively improve the ability of LLMs, which are trained on sequences with a few thousand tokens, to process extremely long sequences. In the future, we will explore efficient training of the context memory module to further enhance the model performance. Besides, combining the key-value cache compression methods with InfLLM can reduce the computational and memory costs for memory management. We hope InfLLM can boost the development of streaming applications of LLMs.

Broader Impact

This paper presents work whose goal is to advance the field of large language models. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Ainslie, J., Ontañón, S., Alberti, C., Cvicek, V., Fisher, Z., Pham, P., Ravula, A., Sanghai, S., Wang, Q., and Yang, L. ETC: encoding long and structured inputs in transformers. In *Proceedings of EMNLP*, pp. 268–284, 2020.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. Longbench: A bilingual, multitask benchmark for long context understanding. *CoRR*, abs/2308.14508, 2023.

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer. *CoRR*, abs/2004.05150, 2020.

Bertsch, A., Alon, U., Neubig, G., and Gormley, M. R. Unlimiformer: Long-range transformers with unlimited length input. *CoRR*, abs/2305.01625, 2023.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N. S., Chen, A. S., Creel, K., Davis, J. Q., Demszky, D., Donahue, C., Doumbouya, M., Durmus, E., Ermon, S., Etchemendy, J., Ethayarajh, K., Fei-Fei, L., Finn, C., Gale, T., Gillespie, L., Goel, K., Goodman, N. D., Grossman, S., Guha, N., Hashimoto, T., Henderson, P., Hewitt, J., Ho, D. E., Hong, J., Hsu, K., Huang, J., Icard, T., Jain, S., Jurafsky, D., Kalluri, P., Karamcheti, S., Keeling, G., Khani, F., Khattab, O., Koh, P. W., Krass, M. S., Krishna, R., Kuditipudi, R., and et al. On the opportunities and risks of foundation models. *CoRR*, abs/2108.07258, 2021.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *Proceedings of NeurIPS*, 2020.

Chen, G., Li, X., Meng, Z., Liang, S., and Bing, L. CLEX: continuous length extrapolation for large language models. *CoRR*, abs/2310.16450, 2023a. doi: 10.48550/ARXIV.2310.16450.

Chen, S., Wong, S., Chen, L., and Tian, Y. Extending context window of large language models via positional interpolation. *CoRR*, abs/2306.15595, 2023b.

Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality, March 2023.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J. G., Le, Q. V., and Salakhutdinov, R. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of ACL*, pp. 2978–2988. Association for Computational Linguistics, 2019.

Dai, Z., Lai, G., Yang, Y., and Le, Q. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. In *Proceedings of NeurIPS*, 2020.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. *CoRR*, abs/2307.08691, 2023.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Proceedings of NeurIPS*, 2022.

Dong, Z., Tang, T., Li, J., and Zhao, W. X. A survey on long text modeling with transformers. *CoRR*, abs/2302.14502, 2023.

Driess, D., Xia, F., Sajjadi, M. S. M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., Huang, W., Chebotar, Y., Sermanet, P., Duckworth, D.,

Levine, S., Vanhoucke, V., Hausman, K., Toussaint, M., Greff, K., Zeng, A., Mordatch, I., and Florence, P. Palm-e: An embodied multimodal language model. In *Proceedings of ICML*, volume 202 of *Proceedings of Machine Learning Research*, pp. 8469–8488. PMLR, 2023.

Goyal, S., Choudhury, A. R., Raje, S., Chakaravarthy, V. T., Sabharwal, Y., and Verma, A. Power-bert: Accelerating BERT inference via progressive word-vector elimination. In *Proceedings of ICML*, volume 119, pp. 3690–3699, 2020.

Graves, A., Wayne, G., and Danihelka, I. Neural turing machines. *CoRR*, abs/1410.5401, 2014.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *CoRR*, abs/2312.00752, 2023.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *Proceedings of ICLR*. OpenReview.net, 2022.

Han, C., Wang, Q., Xiong, W., Chen, Y., Ji, H., and Wang, S. Lm-infinite: Simple on-the-fly length generalization for large language models. *CoRR*, abs/2308.16137, 2023.

Han, X., Zhang, Z., Ding, N., Gu, Y., Liu, X., Huo, Y., Qiu, J., Yao, Y., Zhang, A., Zhang, L., Han, W., Huang, M., Jin, Q., Lan, Y., Liu, Y., Liu, Z., Lu, Z., Qiu, X., Song, R., Tang, J., Wen, J., Yuan, J., Zhao, W. X., and Zhu, J. Pre-trained models: Past, present and future. *AI Open*, 2: 225–250, 2021.

Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., Chen, K., Dong, Y., and Wang, Y. Flashdecoding++: Faster large language model inference on gpus. *CoRR*, abs/2311.01282, 2023.

Huang, Y., Xu, J., Jiang, Z., Lai, J., Li, Z., Yao, Y., Chen, T., Yang, L., Xin, Z., and Ma, X. Advancing transformer architecture in long-context large language models: A comprehensive survey. *CoRR*, abs/2311.12351, 2023.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de Las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b. *CoRR*, abs/2310.06825, 2023.

Jin, H., Han, X., Yang, J., Jiang, Z., Liu, Z., Chang, C., Chen, H., and Hu, X. LLM maybe longlm: Self-extend LLM context window without tuning. *CoRR*, abs/2401.01325, 2024.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of ICML*, volume 119, pp. 5156–5165. PMLR, 2020.

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M. Generalization through memorization: Nearest neighbor language models. In *Proceedings of ICLR*. OpenReview.net, 2020.

Kitaev, N., Kaiser, L., and Levskaya, A. Reformer: The efficient transformer. In *Proceedings of ICLR*. OpenReview.net, 2020.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of SOSP*, pp. 611–626. ACM, 2023.

Lewis, P. S. H., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of NeurIPS*, 2020.

Li, D., Shao, R., Xie, A., Sheng, Y., Zheng, L., Gonzalez, J. E., Stoica, I., Ma, X., , and Zhang, H. How long can open-source llms truly promise on context length?, June 2023.

Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A. Code as policies: Language model programs for embodied control. In *Proceedings of ICRA*, pp. 9493–9500. IEEE, 2023.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *CoRR*, abs/2307.03172, 2023.

Miller, A. H., Fisch, A., Dodge, J., Karimi, A., Bordes, A., and Weston, J. Key-value memory networks for directly reading documents. In *Proceedings of EMNLP*, pp. 1400–1409. The Association for Computational Linguistics, 2016.

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., Jiang, X., Cobbe, K., Eloundou, T., Krueger, G., Button, K., Knight, M., Chess, B., and Schulman, J. Webgpt: Browser-assisted question-answering with human feedback. *CoRR*, abs/2112.09332, 2021.

OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In *Proceedings of NeurIPS*, 2022.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In *Proceedings of UIST*, pp. 2:1–2:22. ACM, 2023.

Peng, B., Quesnelle, J., Fan, H., and Shippole, E. Yarn: Efficient context window extension of large language models. *CoRR*, abs/2309.00071, 2023.

Press, O., Smith, N. A., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *Proceedings of ICLR*. OpenReview.net, 2022.

Qian, C., Cong, X., Yang, C., Chen, W., Su, Y., Xu, J., Liu, Z., and Sun, M. Communicative agents for software development. *CoRR*, abs/2307.07924, 2023.

Rae, J. W., Potapenko, A., Jayakumar, S. M., Hillier, C., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. In *Proceedings of ICLR*. OpenReview.net, 2020.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21:140:1–140:67, 2020.

Shazeer, N. Fast transformer decoding: One write-head is all you need. *CoRR*, abs/1911.02150, 2019.

Su, J. Rectified rotary position embeddings, 2023.

Su, J., Lu, Y., Pan, S., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *CoRR*, abs/2104.09864, 2021.

Sukhbaatar, S., Szlam, A., Weston, J., and Fergus, R. End-to-end memory networks. In *Proceedings of NeurIPS*, pp. 2440–2448, 2015.

Sun, Y., Dong, L., Patra, B., Ma, S., Huang, S., Benhaim, A., Chaudhary, V., Song, X., and Wei, F. A length-extrapolatable transformer. In *Proceedings of ACL*, pp. 14590–14604. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.ACL-LONG.816.

Tay, Y., Dehghani, M., Bahri, D., and Metzler, D. Efficient transformers: A survey. *ACM Comput. Surv.*, 55(6):109:1–109:28, 2023.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023a.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton-Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023b.

Tworkowski, S., Staniszewski, K., Pacek, M., Wu, Y., Michalewski, H., and Milos, P. Focused transformer: Contrastive training for context scaling. *CoRR*, abs/2307.03170, 2023.

Wang, S., Li, B. Z., Khabsa, M., Fang, H., and Ma, H. Linformer: Self-attention with linear complexity. *CoRR*, abs/2006.04768, 2020.

Weston, J., Chopra, S., and Bordes, A. Memory networks. In *Proceedings of ICLR*, 2015.

Wu, Y., Rabe, M. N., Hutchins, D., and Szegedy, C. Memorizing transformers. In *Proceedings of ICLR*. OpenReview.net, 2022.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *CoRR*, abs/2309.17453, 2023.

Xiong, W., Liu, J., Molybog, I., Zhang, H., Bhargava, P., Hou, R., Martin, L., Rungta, R., Sankararaman, K. A., Oguz, B., Khabsa, M., Fang, H., Mehdad, Y., Narang, S., Malik, K., Fan, A., Bhosale, S., Edunov, S., Lewis, M., Wang, S., and Ma, H. Effective long-context scaling of foundation models. *CoRR*, abs/2309.16039, 2023.

Xu, P., Ping, W., Wu, X., McAfee, L., Zhu, C., Liu, Z., Subramanian, S., Bakhturina, E., Shoeybi, M., and Catanzaro, B. Retrieval meets long context large language models. *CoRR*, abs/2310.03025, 2023.

Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontañón, S., Pham, P., Ravula, A., Wang, Q., Yang, L., and Ahmed, A. Big bird: Transformers for longer sequences. In *Proceedings of NeurIPS*, 2020.

Zhang, X., Chen, Y., Hu, S., Wu, Q., Chen, J., Xu, Z., Dai, Z., Han, X., Wang, S., Liu, Z., and Sun, M. Infinitebench: 128k long-context benchmark for language models, 2023a.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C. W., Wang, Z., and Chen, B. H_2o: Heavy-hitter oracle for efficient generative inference of large language models. *CoRR*, abs/2306.14048, 2023b.

Zhao, G., Lin, J., Zhang, Z., Ren, X., Su, Q., and Sun, X. Explicit sparse transformer: Concentrated attention through explicit selection. *CoRR*, abs/1912.11637, 2019.

Zhao, L., Feng, X., Feng, X., Qin, B., and Liu, T. Length extrapolation of transformers: A survey from the perspective of position encoding. *CoRR*, abs/2312.17044, 2023.